cmd\kademliademo\main.go

```go
package main

import (
    "crypto/rand"
    "flag"
    "fmt"
    "log"
    "math/big"
    "sort"
    "strings"

    "github.com/ethereum/go-ethereum/cmd/internal/kademlia"
)

// 이 프로그램은 순수 메모리 상에서 Kademlia 라우팅 테이블과 탐색 절차를 재현해볼 수 있는
// 학습용 데모다. 파라미터로 네트워크 크기, 부트스트랩 노드 수, 병렬 탐색도(α) 등을 조절한다.

func main() {
    // 커맨드라인 옵션으로 네트워크 구성과 탐색 전략을 손쉽게 바꿀 수 있게 한다.
    totalNodes := flag.Int("nodes", 64, "total number of nodes in the simulated network (excluding local)")
    bootstrapCount := flag.Int("bootstrap", 8, "number of bootstrap nodes known initially")
    alpha := flag.Int("alpha", 3, "parallel lookups per iteration")
    bucketSize := flag.Int("k", 16, "bucket size (Kademlia k)")
    targetHex := flag.String("target", "", "hex-encoded target node ID (optional)")
    flag.Parse()

    // 부트스트랩 목록이 전체 노드 수보다 많을 수는 없으니 사전에 체크한다.
    if *bootstrapCount > *totalNodes {
        log.Fatalf("bootstrap (%d) cannot exceed total nodes (%d)", *bootstrapCount, *totalNodes)
    }

    // 로컬 노드 ID를 무작위로 만들고, 그 ID를 중심으로 테이블을 초기화한다.
    localID := randomID()
    table := kademlia.NewTable(localID, *bucketSize)

    // 시뮬레이션에 사용할 가상 노드들을 생성한 뒤, 일부를 부트스트랩 노드로 등록한다.
    nodes := generateNodes(*totalNodes)
    for i := 0; i < *bootstrapCount; i++ {
        table.AddNode(nodes[i])
    }

    // 사용자가 목표 ID를 직접 지정하지 않으면 중간 위치의 노드를 타깃으로 삼는다.
    targetID := chooseTarget(*targetHex, nodes)
    sim := simulator{nodes: nodes, bucketSize: *bucketSize}

    // 현재 세팅을 눈으로 확인할 수 있도록 요약 정보를 출력한다.
    fmt.Printf("Local node: %s\n", shortID(localID))
    fmt.Printf("Bootstrap nodes: %s\n", joinLabels(nodes[:*bootstrapCount]))
    fmt.Printf("Target ID: %s\n\n", shortID(targetID))
```

```go

    // Kademlia iterative lookup을 실행하고, 각 단계별로 어떤 노드를 물어봤는지와 수확한 노드를 저장한다.
    report := iterativeLookup(table, sim, targetID, *alpha, *bucketSize)

    fmt.Println("Lookup trace:")
    for i, step := range report.Steps {
        fmt.Printf("  Step %d: queried %s, discovered %s\n", i+1, joinLabels(step.Queried), joinLabels(step.Discovered))
    }
    fmt.Println()

    fmt.Println("Routing table buckets (non-empty):")
    infos := table.Buckets()
    for _, info := range infos {
        fmt.Printf("  Bucket %3d: %d nodes\n", info.Index, info.Count)
    }
    fmt.Println()

    closest := table.FindnodeByID(targetID, *bucketSize)
    fmt.Printf("Closest %d nodes to target:\n", len(closest))
    for i, node := range closest {
        fmt.Printf("  %2d. %s (distance=%s)\n", i+1, nodeString(node), distanceString(node.ID, targetID))
    }
}

// lookupReport는 탐색 과정에서 각 단계별로 어떤 노드를 질의하고 어떤 노드를 발견했는지를 담는다.
type lookupReport struct {
    Steps []lookupStep
}

// lookupStep은 하나의 라운드에서 질의한 노드와 그 결과로 얻은 응답을 표현한다.
type lookupStep struct {
    Queried    []kademlia.Node
    Discovered []kademlia.Node
}

// lookupEntry는 후보 목록을 관리하면서 어떤 노드를 이미 질의했는지 추적하기 위해 사용한다.
type lookupEntry struct {
    node    kademlia.Node
    queried bool
}

// simulator는 실제 네트워크 대신 정렬된 결과를 돌려주는 순수 함수형 모형으로, Kademlia의 거리 기반 응답을 흉내 낸다.
type simulator struct {
    nodes      []kademlia.Node
    bucketSize int
}

// Query는 타깃에 가까운 노드들을 bucketSize(K) 개 이내로 정렬해 반환한다.
func (sim simulator) Query(target kademlia.ID, max int) []kademlia.Node {
```

```go
    if max <= 0 {
        max = sim.bucketSize
    }
    sorted := append([]kademlia.Node(nil), sim.nodes...)
    sort.Slice(sorted, func(i, j int) bool {
        return compareDistance(sorted[i].ID, sorted[j].ID, target) < 0
    })
    if len(sorted) > max {
        sorted = sorted[:max]
    }
    return cloneNodes(sorted)
}

// iterativeLookup은 Kademlia의 α 병렬 탐색 전략을 그대로 흉내 내면서 테이블을 갱신하고 추적 기록을 만든다.
func iterativeLookup(table *kademlia.Table, sim simulator, target kademlia.ID, alpha, bucketSize int) lookupReport {
    shortlist := make([]lookupEntry, 0)
    seen := map[kademlia.ID]bool{table.ID(): true}

    addNode := func(n kademlia.Node) {
        if seen[n.ID] {
            return
        }
        seen[n.ID] = true
        shortlist = append(shortlist, lookupEntry{node: n})
    }

    for _, n := range table.FindnodeByID(target, bucketSize) {
        addNode(n)
    }

    report := lookupReport{}
    for {
        toQuery := selectNextBatch(shortlist, target, alpha)
        if len(toQuery) == 0 {
            break
        }
        step := lookupStep{}
        for _, idx := range toQuery {
            if shortlist[idx].queried {
                continue
            }
            shortlist[idx].queried = true
            queriedNode := shortlist[idx].node
            step.Queried = append(step.Queried, queriedNode)

            response := sim.Query(target, bucketSize)

            var discovered []kademlia.Node
            for _, resp := range response {
                if resp.ID == queriedNode.ID {
                    continue
                }
```

```go
                table.AddNode(resp)
                if !seen[resp.ID] {
                    addNode(resp)
                    discovered = append(discovered, resp)
                }
            }
            step.Discovered = append(step.Discovered, discovered...)
        }
        report.Steps = append(report.Steps, step)
    }
    return report
}

// selectNextBatch는 아직 질의하지 않은 후보들 중에서 타깃에 가장 가까운 α개를 골라낸다.
func selectNextBatch(entries []lookupEntry, target kademlia.ID, alpha int) []int {
    type candidate struct {
        idx  int
        node kademlia.Node
    }
    cand := make([]candidate, 0)
    for i, entry := range entries {
        if entry.queried {
            continue
        }
        cand = append(cand, candidate{idx: i, node: entry.node})
    }
    sort.Slice(cand, func(i, j int) bool {
        return compareDistance(cand[i].node.ID, cand[j].node.ID, target) < 0
    })
    if len(cand) > alpha {
        cand = cand[:alpha]
    }
    idxs := make([]int, len(cand))
    for i, c := range cand {
        idxs[i] = c.idx
    }
    return idxs
}

// generateNodes는 시뮬레이터용 가상의 노드 ID와 라벨을 만들어낸다.
func generateNodes(n int) []kademlia.Node {
    nodes := make([]kademlia.Node, n)
    for i := 0; i < n; i++ {
        nodes[i] = kademlia.Node{
            ID:    randomID(),
            Label: fmt.Sprintf("N%02d", i+1),
        }
    }
    return nodes
}

// randomID는 256비트 랜덤 ID를 생성해 Kademlia ID 공간에 새 노드를 배치한다.
func randomID() kademlia.ID {
    var id kademlia.ID
```

```go
    if _, err := rand.Read(id[:]); err != nil {
        panic(err)
    }
    return id
}

// compareDistance는 Kademlia에서 사용되는 XOR 거리를 문자열 비교로 구현해 정렬에 활용한다.
func compareDistance(a, b, target kademlia.ID) int {
    da := xorBytes(a, target)
    db := xorBytes(b, target)
    return strings.Compare(da, db)
}

// xorBytes는 두 ID의 XOR 결과를 문자열로 바꿔 거리 비교를 돕는다.
func xorBytes(a, b kademlia.ID) string {
    ab := a.Bytes()
    bb := b.Bytes()
    out := make([]byte, len(ab))
    for i := range ab {
        out[i] = ab[i] ^ bb[i]
    }
    return string(out)
}

// shortID는 긴 ID를 터미널에서 보기 좋게 앞부분만 잘라낸다.
func shortID(id kademlia.ID) string {
    return id.TerminalString()
}

// joinLabels는 여러 노드의 라벨을 한 줄로 보여주기 위해 쉼표로 묶는다.
func joinLabels(nodes []kademlia.Node) string {
    if len(nodes) == 0 {
        return "(none)"
    }
    labels := make([]string, len(nodes))
    for i, n := range nodes {
        if n.Label != "" {
            labels[i] = n.Label
        } else {
            labels[i] = shortID(n.ID)
        }
    }
    return strings.Join(labels, ", ")
}

// cloneNodes는 정렬 과정에서 원본 슬라이스를 건드리지 않기 위해 복사본을 만든다.
func cloneNodes(nodes []kademlia.Node) []kademlia.Node {
    cloned := make([]kademlia.Node, len(nodes))
    copy(cloned, nodes)
    return cloned
}

// chooseTarget은 사용자가 직접 지정한 타깃 ID가 있으면 그 값을, 없으면 중앙 노드를 선택한다.
func chooseTarget(hex string, nodes []kademlia.Node) kademlia.ID {
```

```go
    if hex == "" {
        idx := len(nodes) / 2
        return nodes[idx].ID
    }
    bigInt, ok := new(big.Int).SetString(hex, 16)
    if !ok {
        log.Fatalf("invalid hexadecimal target ID")
    }
    bytes := bigInt.Bytes()
    var id kademlia.ID
    copy(id[32-len(bytes):], bytes)
    return id
}

// nodeString은 노드 라벨과 ID를 함께 출력해 사람이 구분하기 쉽게 만든다.
func nodeString(n kademlia.Node) string {
    label := n.Label
    if label == "" {
        label = shortID(n.ID)
    }
    return fmt.Sprintf("%s (%s)", label, n.ID)
}

// distanceString은 두 ID의 XOR 결과를 16진수로 출력해 거리 감각을 돕는다.
func distanceString(a, b kademlia.ID) string {
    distance := make([]byte, len(a))
    for i := range distance {
        distance[i] = a[i] ^ b[i]
    }
    return fmt.Sprintf("0x%x", distance)
}

```